

流程通用配置能力

前期分析

背景简介

US需要在每个接口获取并给端返回uiAB实验的素材配置；

端侧的诉求是一次所有接口生效，避免跟单接口迭代去增加此能力。

目标收益

- 给端提供自适应的uiAB实验的素材数据
- 一次配置永久有效，后续新增接口无需单独设置

需求分析

功能需求

1. 提供通用的流程配置能力
2. 可以在一个位置配置所有流程生效的Node

技术需求

- 兼容现有框架逻辑
 - config配置兼容
 - 运行兼容
- 兼容现有业务结果
 - Response格式
 - Map格式

实现分析

前期调研

产品调研

无

技术调研

从需求来看，目前业务中已实现了此逻辑的Node（RuleConfigNodeService），此Node逻辑如下：

1. 根据入参构造查询的ComponentKeyList列表
2. 根据上步的列表循环调用运营域方法查询素材数据（目前仅uiABConfig）
 - a. 构造查询request
 - b. 查询素材Diamond配置
 - c. 过滤素材结果
3. 打印Monitor监控日志
4. 返回结果Map

此外根据沟通可知，此逻辑将来有可能进行迭代变更。

综上，可见此Node是一个较为通用的业务Node，它适合被所有流程集成，但逻辑需保留在业务服务中进行维护。

实现这样的逻辑可有如下方案：

- 方案一：在业务服务中抽取CommonProcessConfigBuilder，改造所有的Process代码继承其
- 方案二：改造ProcessConfig使其具有全局配置的能力，业务中可在一处配置上文Node为全局Node
- 方案三：改造Node的配置参数，增加全局Node标识，开启后所有Process默认集成它

差异分析

产品差异

无

技术差异

从上述三个方案分析

分析	方案一	方案二	方案三
优点	• 无需动框架，开发成本最低	• 业务代码只需一处修改 • 可兼容扩展其他配置 ◦ 线程 ◦ 后处理 ◦ ...	• 业务代码只需一处修改

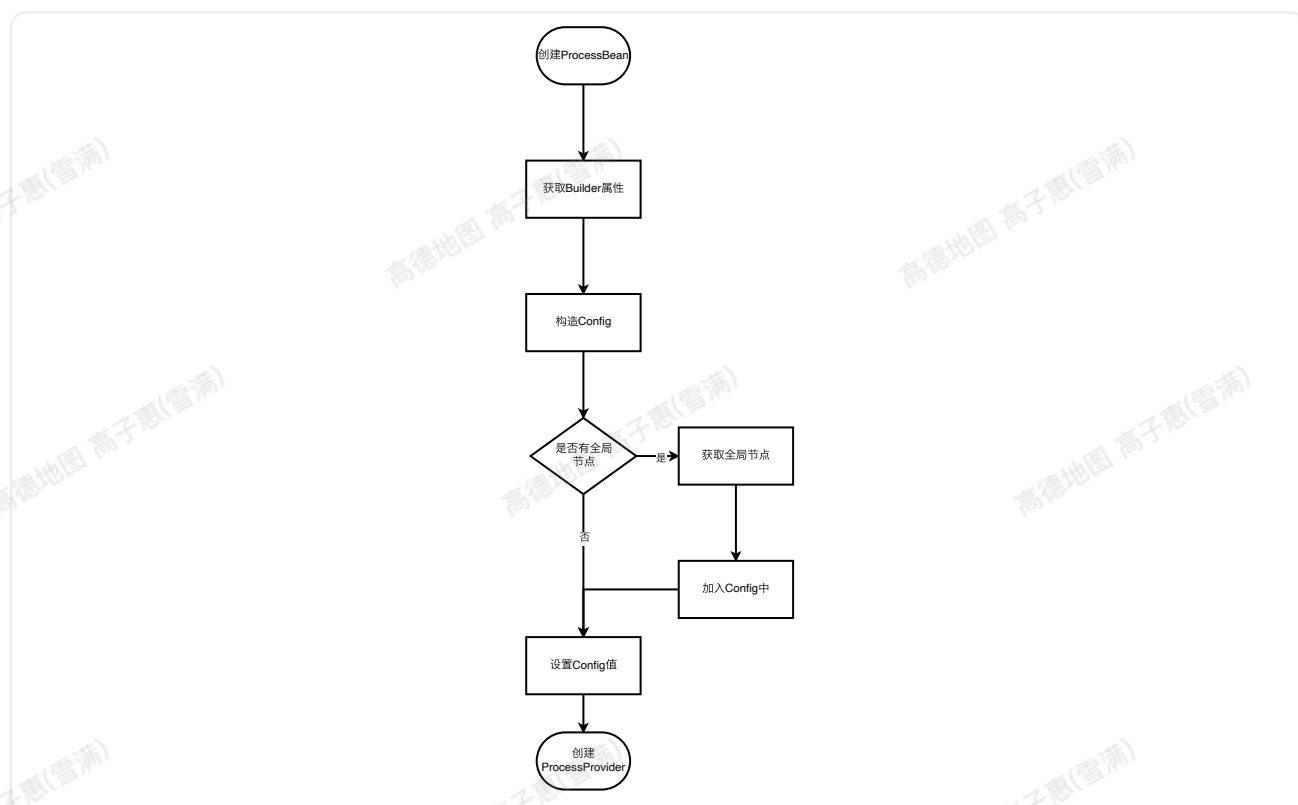
缺点	每个接口需要明文集成公共方法，覆盖度和适用性都不够	和业务流程定义分离	- 从下层影响上层，不合理 - 和业务流程定义分离
----	---	---	--

实现结论

综上可见方案二明显更为合理，故采用方案二

方案设计

整体方案



关键方案

响应结果兼容

- Response类型响应兼容：将现有Node结果字段移动到CommonResponse中
- Map类型响应兼容：目前分析代码Map结构会根据ProcessConfig自动收集，一般无需做特殊处理，只有如search这种子流程嵌套的复杂结构需要额外处理结果map，当前已处理过；后续再有需单独处理的流程再考虑

Node顺序兼容

不同流程的Node顺序和配置各异，因此考虑将全局配置拆分为头部节点配置和尾部节点配置，分别在构造的前后追加节点，不再在业务节点中追加，避免对原有流程造成影响。

详细设计

```
public ProcessConfig build() { ① 克生 2
    if (this.nodes == null || this.nodes.isEmpty()) {
        throw new RegistryException("ProcessConfig build error nodes is null");
    }
    List<? extends NodeConfig> nodeList = this.nodes.stream().map(BaseNodeConfigBuilder::build).collect(Collectors.toList());
    long count = nodeList.stream().map(NodeConfig::getAlias).filter(StringUtils::isNotBlank).count();
    long disCount = nodeList.stream().map(NodeConfig::getAlias).filter(StringUtils::isNotBlank).distinct().count();
    if (disCount != count) {
        throw new RegistryException("ProcessConfig build error alias exit duplicate. name:" + name);
    }
    if (this.request == null) {
        throw new RegistryException("ProcessConfig build error request is null. name:" + name);
    }
    if (this.response == null) {
        throw new RegistryException("ProcessConfig build error response is null. name:" + name);
    }
    return createProcessConfig(nodeList);
}

@NotNull 1个用法 ① 克生 2
private ProcessConfig createProcessConfig(List<? extends NodeConfig> nodeList) {
    ProcessConfig processConfig = new ProcessConfig(this.name, nodeList);
    processConfig.setDesc(this.desc);
    processConfig.setRequest(this.request);
    processConfig.setResponse(this.response);
    processConfig.setExecutorServiceSupplier(executorServiceSupplier);
    processConfig.setHsfServiceVersion(this.hsfServiceVersion);
    processConfig.setHsfServiceGroup(this.hsfServiceGroup);
    processConfig.setHsfServiceCorePoolSize(this.hsfServiceCorePoolSize);
    processConfig.setHsfServiceMaxPoolSize(this.hsfServiceMaxPoolSize);
    processConfig.setHsfServiceTimeout(this.hsfServiceTimeout);
    processConfig.setPostJoinPoint(this.postJoinPoint);
    processConfig.setAutoResponseAssemble(this.autoResponseAssemble);
    processConfig.setScope(this.scope);
    processConfig.setMetricsCalculator(this.metricsCalculator);
    return processConfig;
}
```

```
@Data ① 王业雷(不显) 3
public class ProcessConfig {

    public ProcessConfig(final String name, final List<? extends NodeConfig> nodes) { ① shanchong 1
        this.name = name;
        this.nodes = nodes;
    }

    private String name;

    private List<? extends NodeConfig> nodes;

    private String desc;

    private Class<? extends Request> request;

    private Class<? extends Response> response;
}
```

在ProcessConfig构造时将全局配置中的Node加入到Config中

复制代码

```

/**
 * 静态全局配置类
 */
@Data
public static class Global {
    /**
     * 头部节点，会在所有流程头部添加
     */
    private List<? extends NodeConfig> headNodes;
    /**
     * 尾部节点，会在所有流程尾部添加
     */
    private List<? extends NodeConfig> tailNodes;

    /**
     * 线程池
     */
    private Supplier<ExecutorService> executorServiceSupplier;

    /**
     * 唯一单例
     */
    private static final Global INSTANCE = new Global();

    private Global() {
        // 私有构造
    }
}

```

其他

成本评估

- GBF框架开发 0.5d
 - 开发实现
 - 自测用例
- US业务服务适配开发 0.5d
 - 开发实现
 - 自测用例

测试建议

开发关注：错误日志和线程池监控

测试方案：原有UI实验逻辑回归，所有接口diff，已实现UI实验功能的接口重点diff

重点diff接口：

- assembly
- **search**
- **fillOrder**
- **poiDetail**
- **retainPortal**
- contentEditor
- nearBy
- orderDetail
- portal
- productDetail
- productList
- recommend

实施计划

☒ 10.14号QA进入

☒ 线程池正常

线程-任务-全机房-gray

区间查看

时间对比

机房对比

分组对比

采样信息

时间：

3分钟

10分钟

1小时

2小时

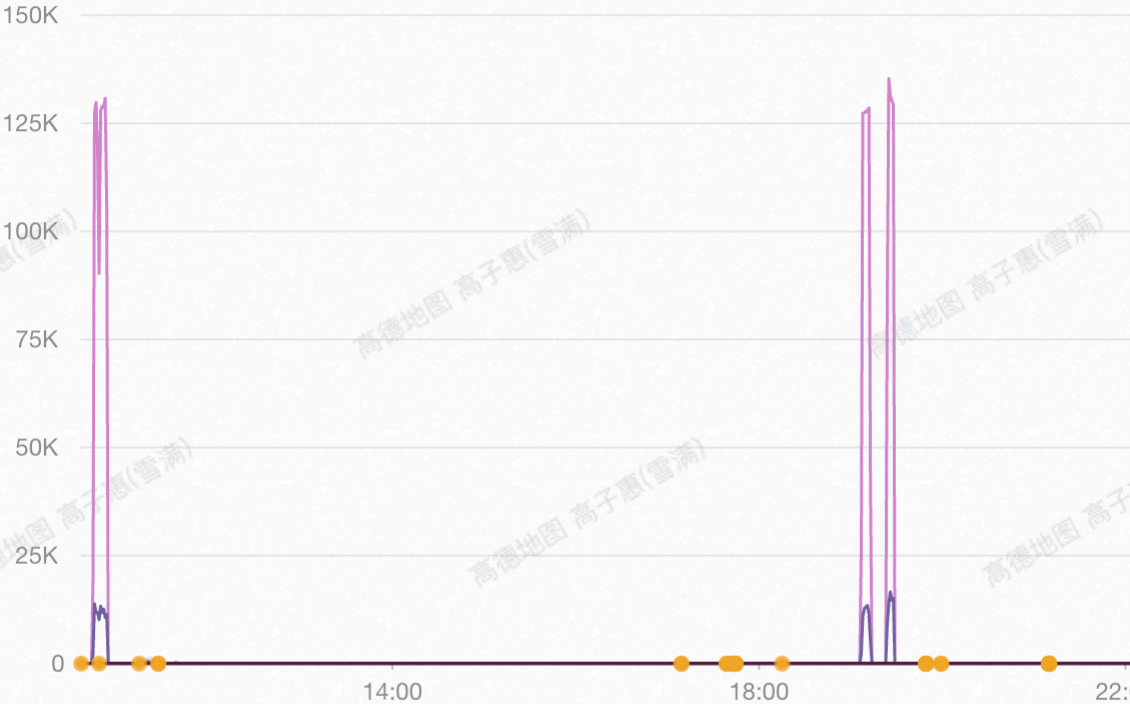
3小时

1天

自定义

名称：

全部



☒ 错误日志正常

biz-error_uni

查看索引 高级功能

@app_group:lse2-us-business-service_gray9

昨天(相对) 查询

No data to display

日志总条数：0 查询状态：结果精确

☒ 原逻辑回归正常

☒ 10.17号上线

遗留内容

•

后续规划

-

附录

名词解释

参考文档

Response改造：[📄 接口结果强定义改造](#)